中间件

教学目标

1、理解中间件的核心概念

2、掌握自定义中间件的开发方法

3、理解中间件的执行顺序规则

4、掌握CSRF跨站请求伪造的原理与防护

5、掌握importlib模块的动态导入功能

一、中间件

在Django中，中间件（Middleware）是一个轻量级的、底层的插件系统，用于全局改变Django的输入（请求）或输出（响应）。每个中间件组件负责完成一些特定的功能，例如：处理请求、处理响应、处理异常等。

中间件的结构：
中间件是一个可调用的对象，可以是一个函数（通常称为中间件函数）或一个类（通常称为中间件类）。Django提供了一种类形式的中间件，更常用。

流程图:

django 还支持自定义中间件且暴露了5个可以自定义的方法:

- process_request（掌握)
- process_response (掌握)
- process_view
- process_template_response
- process_exception

自定义中间件的步骤:

```
1. 在根目录或者应用名下创建一个任意名称的文件夹
2. 在文件夹下面创建一个任意名称的 py文件
3. 在 Py文件中定义类,必须继承MiddlewareMixin 类
4. 之后就可以定义五个方法(需要几个写几个)
5. 将类的路径以字符串的形式注册到配置文件中
```

app01 -> mymiddleware

创建文件夹

app01 -> mymiddleware->mymd.py

创建文件

mymd.py

```
class MyMiddleware(MiddlewareMixin):
    #在访问视图函数之前进入到这个方法
    def process_request(self,request):
        print('这是第一个自定义中间件中的precess_request方法')
        #如果该方法返回了一个HttpResponse 对象,那么请求会直接原路返回,不在继续执行
        #return HttpResponse('这是process_request的中间件')

    #request参数 因为是返回的时候也有可能带请求的参数出去
    #response参数 返回的响应
    def process_response(self,request,response):
        print('这是第一个自定义中间件中的process_response方法')
        return response

class MyMiddleware2(MiddlewareMixin):
    # 在访问视图函数之前进入到这个方法
    def process_request(self, request):
        print('这是第二个自定义中间件中的precess_request方法')

    def process_response(self, request, response):
        print('这是第二个自定义中间件中的process_response方法')
        return response
```

settings.py

```
MIDDLEWARE = [
    ........
    #注册
    'app01.mymiddleware.mymd.MyMiddleware',
    'app01.mymiddleware.mymd.MyMiddleware2'
]
```

访问路由测试

这是第一个自定义中间件中的precess_request方法
这是第二个自定义中间件中的precess_request方法
我是登录页面
这是第二个自定义中间件中的process_response方法
这是第一个自定义中间件中的process_response方法

process_request

- 请求来的时候需要经过每一个中间件的process_request方法.按照配置文件中注册的中间件从上往下的顺序依次执行.如果中间件没有定义该跳过,执行下一个中间件

process_response

- 响应走的时候需要经过每一个中间件的process_response 方法,该方法有两个额外的参数:request,response.
- 该方法必须返回一个HttpResponse 对象,默认返回的形参response 接受到的就是视图函数的返回值
- 也可以自己定义返回内容,那么后续中间件拿到的内容都是自定义的返回内容

注意:执行顺序是按照配置文件中注册的中间件从下往上执行.如果中间件没有定义该方法,会跳过执行下一个中间件

process_view

路由匹配成功之后执行视图函数之前自动执行中间件中的该方法.按照配置文件中的顺序从上往下执行

```
class MyMiddleware(MiddlewareMixin):
    .....


    def process_view(self,request,view_name,*args,**kwargs):
        print(view_name)
        print(args)
        print(kwargs)
        print('这是第一个自定义中间件中的process_view方法')

class MyMiddleware2(MiddlewareMixin):
    .....

    def process_view(self,request,view_name,*args,**kwargs):
        print(view_name)
        print(args)
        print(kwargs)
        print('这是第二个自定义中间件中的process_view方法')
```

```
这是第一个自定义中间件中的precess_request方法
这是第二个自定义中间件中的precess_request方法
<function login at 0x000001A84A50DBD0>
((), {})
{}
这是第一个自定义中间件中的process_view方法
<function login at 0x000001A84A50DBD0>
((), {})
{}
这是第二个自定义中间件中的process_view方法
我是登录页面
这是第二个自定义中间件中的process_response方法
这是第一个自定义中间件中的process_response方法
```

总结：【路由匹配后，视图执行前】

process_template_response

在视图函数执行完成后被调用 但前提是视图返回的响应对象具有 .render() 方法

mymd.py

```
class MyMiddleware(MiddlewareMixin):
    #在访问视图函数之前进入到这个方法
    def process_request(self,request):
        print('这是第一个自定义中间件中的precess_request方法')
        #如果该方法返回了一个HttpResponse 对象,那么请求会直接原路返回,不在继续执行
        #return HttpResponse('这是process_request的中间件')

    #request参数 因为是返回的时候也有可能带请求的参数出去
    #response参数 返回的响应
    # 在访问视图函数之后进入到这个方法
    def process_response(self,request,response):
        print('这是第一个自定义中间件中的process_response方法')
        return response

    #在路由匹配后，视图执行前
    def process_view(self,request,view_name,*args,**kwargs):
        print(view_name)
        print(args)
        print(kwargs)
        print('这是第一个自定义中间件中的process_view方法')

    #在视图函数执行完成后被调用 但前提是视图返回的响应对象具有 .render() 方法

    def process_template_response(self,request,response):
        print('这是第一个自定义中间件中的process_template_response方法')
```

```
        return response
```

urls.py

```
path('aaa/',views.aaa)
```

views.py

```
def aaa(request):
    print('这是视图函数aaa')
    obj = HttpResponse('这是aaa页面')
    def render():
        print('这是内部的render函数')
        return HttpResponse('ok')

    obj.render=render  # 给HttpResponse对象添加一个render的属性
    return obj
```

测试:

```
这是第一个自定义中间件中的precess_request方法
这是第二个自定义中间件中的precess_request方法
<function aaa at 0x000001FEFF7FE3B0>
((), {})
{}
这是第一个自定义中间件中的process_view方法
<function aaa at 0x000001FEFF7FE3B0>
((), {})
{}
这是第二个自定义中间件中的process_view方法
这是视图函数aaa
这是第一个自定义中间件中的process_template_response方法
这是内部的render函数
这是第二个自定义中间件中的process_response方法
这是第一个自定义中间件中的process_response方法
```

使用场景

1. 添加全局模板：例如:在每个模板中都要用到的变量
2. 动态替换模板：根据请求条件更换不同的模板。
3. 修改模板：在渲染前统一添加或修改数据。

process_exception

在视图函数抛出异常时被调用。该方法允许中间件在异常发生时介入处理

```
class MyMiddleware(MiddlewareMixin):
    ........

    #捕捉全局异常
    def process_exception(self, request, exception):
        # 记录异常信息
        print(f"捕获到异常: {exception}")
        # 返回自定义错误页面
        return HttpResponse(f"<h1>服务器错误</h1><p>{exception}</p>", status=500)
```

urls.py

```
path('test_ex/',views.test_ex)
```

app01 -> views.py

```
def test_ex(request):
    # 模拟抛出异常
    raise ValueError("这是一个测试异常！")
```

注意事项

- 只有视图函数（或中间件）中抛出的未处理异常才会触发 `process_exception`
- 如果中间件返回了响应,则后续中间件的 `process_exception` 方法将不再执行
- 该方法对处理全局异常非常有用,例如统一错误日志记录或自定义错误页面

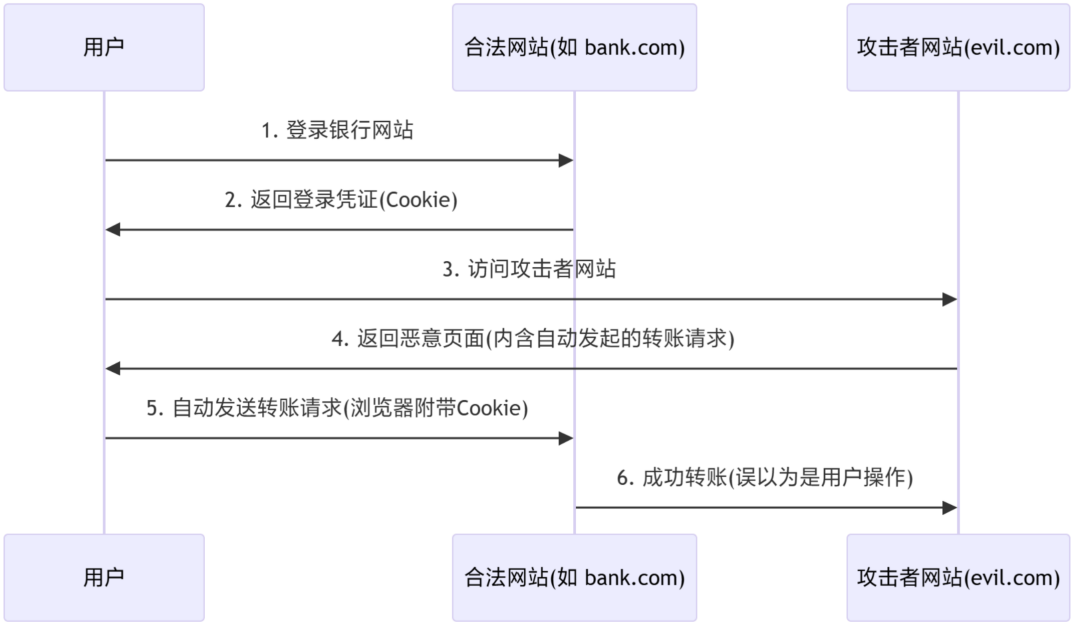
中间件执行顺序规则

1. 请求阶段：按 `MIDDLEWARE` 列表 **从上到下** 执行（`process_request` → `process_view` → 视图)
2. 响应阶段：按 `MIDDLEWARE` 列表 **从下到上** 执行（`process_response` / `process_exception`）

二、csrf 跨站伪造请求

什么是跨站请求伪造（CSRF）

跨站请求伪造（Cross-Site Request Forgery）是一种Web安全漏洞，攻击者利用用户已经登录的身份，在用户不知情的情况下执行非授权的操作。



实现一个钓鱼网址:

- 做一个和正规网站一样的页面,使用 form 表单提交数据
- 收款人是没有 name 属性的 input
- 隐藏一个有 name 属性,并且已经写好的值的input
- 表面上你是向正规网站的账户提交数据,实际上你是向钓鱼网站的账户打钱的.

正规网址

urls.py

```
path('transfer/',views.transfer)
```

app01 -> views.py

```
def transfer(request):
    if request.method =='POST':
        username = request.POST.get('username')
        target_user = request.POST.get('target_user')
        money = request.POST.get('money')
        print(f'{username}给{target_user}转了{money}元')

    return render(request,'transfer.html')
```

transfer.html

```
<h1>这是正规网址</h1>
<form action="" method="post">
    <p>用户: <input type="text" name="username"> </p>
    <p>对方: <input type="text" name="target_user"> </p>
    <p>金额: <input type="text" name="money"> </p>

    <input type="submit">
</form>
```

重新新建一个项目实现

自行创建 端口号:8001

钓鱼网址

urls.py

```
path('transfer/',views.transfer)
```

app01 -> views.py

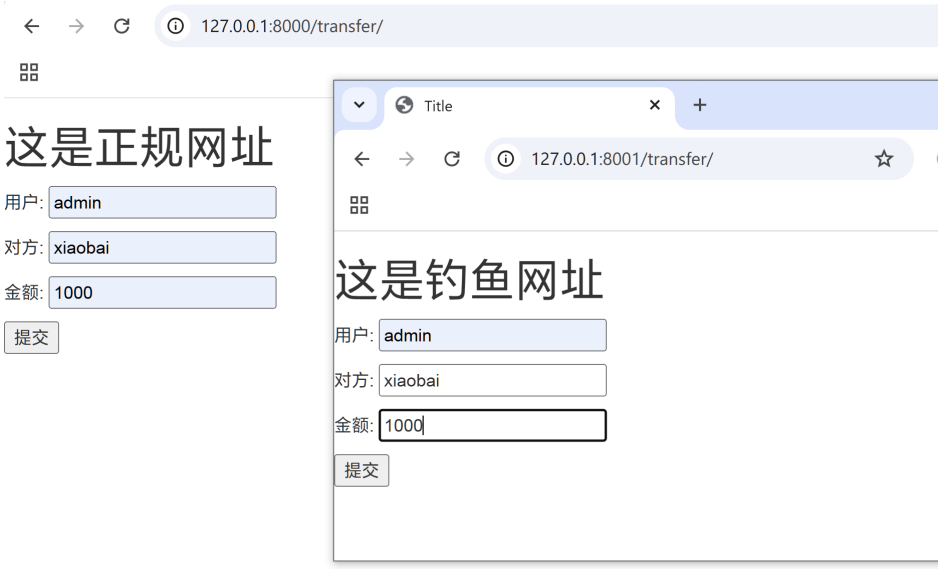
```
def transfer(request):
    return render(request,'transfer.html')
```

transfer.html

```
<h1>这是钓鱼网址</h1>
<form action="" method="post">
    <p>用户: <input type="text" name="username"> </p>
    <p>对方: <input type="text" name="target_user"> </p>
    <p>金额: <input type="text" name="money"> </p>

    <input type="submit">
</form>
```

经过测试发现



> 无论是正规网址还是钓鱼网址 输入的数据都是admin向xiaobai转账
>
> admin给xiaobai转了1000元
>
> admin给zhangsan转了1000元
>
> 结果正规网址收到的是 admin向zhangsan转账了 这样的话就相当于被人篡改了

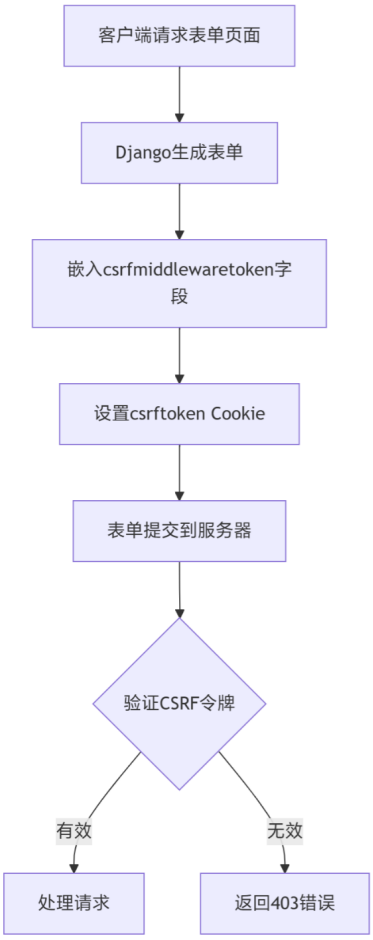
如何规避上述问题?

CSRF保护机制

Django通过内置的`CsrfViewMiddleware`提供全面的CSRF保护:

保护机制核心

1. **CSRF Token生成**:服务端为每个会话生成唯一令牌
2. Token分发
   - 通过Cookie发送给客户端(`csrftoken`)
   - 在表单中嵌入隐藏字段(`csrfmiddlewaretoken`)
3. 请求验证
   - POST请求必须包含有效的CSRF令牌
   - 对比Cookie中的令牌和表单/请求头中的令牌



小总结

网站在给用户返回一个具有提交功能的页面时需要加上一个唯一标识符当页面后端提交 post 请求时,后端会检查唯一标识符,如果标识符不对,就拒绝访问

Django中使用CSRF保护

1. 确保中间件启用

```
# settings.py
MIDDLEWARE = [
    # ...
    'django.middleware.csrf.CsrfViewMiddleware',  # 确保启用
    # ...
]
```

2. 表单模板中使用CSRF

```
<form action="" method="post">
    {% csrf_token %}
    <p>用户: <input type="text" name="username"> </p>
    <p>对方: <input type="text" name="target_user"> </p>
    <p>金额: <input type="text" name="money"> </p>

    <input type="submit">
</form>
```

发现加了csrf_token的时候表单加了一个隐藏字段 value值是一个随机的

```
</head>
<body>
<h1>这是正规网址</h1>
<form action="" method="post">
    <input type="hidden" name="csrfmiddlewaretoken" value="jtxIhMZuNxJ5x8FJ2foLz2zWSdIoTUbVqBOfLXZPmKZmTjI6jysMrDkyfTsMY4Di">
    <p>用户: <input type="text" name="username"> </p>
    <p>对方: <input type="text" name="target_user"> </p>
    <p>金额: <input type="text" name="money"> </p>

    <input type="submit">
</form>


</body>
</html>
```

3. AJAX请求处理CSRF

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>


    {% load static %}

    <!-- 导入外部的css-->
    <link rel="stylesheet" href={% static 'bootstrap-3.4.1-
dist/css/bootstrap.min.css' %}>
    <!-- 导入外部的js-->
    <script src={% static "js/jquery-3.7.0.min.js" %}></script>
    <!-- 导入外部的js-->
    <script src={% static "bootstrap-3.4.1-dist/js/bootstrap.min.js" %}>
</script>

</head>
<body>
<h1>这是正规网址</h1>
<form action="" method="post">
    {% csrf_token %}
    <p>用户: <input type="text" name="username"> </p>
    <p>对方: <input type="text" name="target_user"> </p>
    <p>金额: <input type="text" name="money"> </p>

    <input type="submit">
</form>


<button id="btn">ajax</button>

<script>
    // 第一种: 从Cookie中获取CSRF值
    function getCookie(name) {
        let cookieValue = null;
        if (document.cookie && document.cookie !== '') {
            const cookies = document.cookie.split(';');
            for (let i = 0; i < cookies.length; i++) {
```

```
                const cookie = cookies[i].trim();
                if (cookie.substring(0, name.length + 1) === (name + '=')) {
                    cookieValue =
decodeURIComponent(cookie.substring(name.length + 1));
                    break;
                }
            }
        }
        return cookieValue;
    }

    $('#btn').click(function () {
        $.ajax({
            url:'',
            type:'post',
            data:{
                username:'aaa',
                target_user:'bbb',
                money:1000,
                //第一种
                //csrfmiddlewaretoken: getCookie('csrftoken')
                //第二种
                csrfmiddlewaretoken: '{{ csrf_token }}'
            },
            success:function(res){

            }
        })

    })


</script>

</body>
</html>
```

三、补充模块

importlib

Python 的 `importlib` 模块是用于实现导入（import）功能的模块，提供了丰富的工具

模仿django中的配置文件中的MIDDLEWARE的调用演示

创建项目（自行创建）

lib_test

创建pt文件夹（自行创建）

pt

email.py

```
class Email:

    def send(self,content):
        print('邮箱通知',content)
```

qq.py

```
class Qq:

    def send(self,content):
        print('qq通知',content)
```

wechat.py

```
class Wechat:

    def send(self,content):
        print('微信通知',content)
```

settings.py

```
NOTIFY_LIST=[
    'pt.email.Email',
    'pt.qq.Qq',
    'pt.wechat.Wechat',
]
```

start.py

```
from pt import settings

import importlib


def send_all(content):

    for path_str in settings.NOTIFY_LIST:
        #以.来分割 最多分割1次 得到模块和类名
        #module_path: 'pt.email'
        #class_str : 'Email'
        module_path ,class_str = path_str.rsplit('.',maxsplit=1)
        #import_module # 等价于 from pt import email
        module = importlib.import_module(module_path)
        #getattr(路径,对象名)  --> 调用模块中的对象
        #获取到模块中的类名
        cls = getattr(module,class_str)
        #生成类对象
        obj = cls()
        #调用发送方法
```

```
        obj.send(content)

if __name__ == '__main__':
    send_all('hello')
```

四、作业

完成本天的案例